

Python Interview Cheat Sheet (Easy to Remember)

For Quick Revision | Simple Examples | Core Concepts

1. Python Basics (Quick Recap)

Key Points

1. Interpreted, dynamically typed, OOP, high-level language
1. Indentation (4 spaces) = code blocks
1. Everything is an **object**

Simple Example

```
python
# Variable (no type declaration)
name = "Python"
num = 10

# Function
def add(a, b):
    return a + b

print(add(2,3)) # Output: 5
```

2. Decorators (Function + Class)

What is a Decorator?

1. A function that **adds extra behavior** to another function/class **without modifying original code**
1. Uses `@` syntax (syntactic sugar)

A. Function Decorator

```
python
# Simple decorator: print before/after function call
def my_decorator(func):
    def wrapper():
        print("Before function runs")
        func() # call original function
        print("After function runs")
    return wrapper

# Use decorator
@my_decorator
def greet():
    print("Hello!")

greet()
```

Output:

```
text
Before function runs
Hello!
After function runs
```

Decorator with Arguments

```
python
def decorator(func):
    def wrapper(*args, **kwargs): # accept any args
        print("Start")
        result = func(*args, **kwargs)
        print("End")
        return result
    return wrapper

@decorator
def add(a,b):
    return a+b

print(add(3,4)) # Output: 7
```

B. Class Decorator

Class acts as decorator → uses `__call__` method

```
python
class ClassDecorator:
    def __init__(self, func):
        self.func = func

    def __call__(self, *args, **kwargs):
        print("Class decorator running")
        return self.func(*args, **kwargs)

@ClassDecorator
def say_hi():
    print("Hi from function")

say_hi()
```

Output:

```
text
Class decorator running
Hi from function
```

3. Class Dunder (Magic) Methods

What are Dunders?

1. Double underscore `__method__`
1. Automatically called by Python (e.g., `__init__`, `__str__`)

Most Important Dunders (Easy to Remember)

```
python
class Person:
    # 1. Constructor (called when object created)
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```

# 2. String representation (print(object))
def __str__(self):
    return f"Name: {self.name}, Age: {self.age}"

# 3. Length (len(object))
def __len__(self):
    return self.age

# 4. Add objects (obj1 + obj2)
def __add__(self, other):
    return self.age + other.age

# Usage
p1 = Person("Alice", 25)
p2 = Person("Bob", 30)

print(p1)      # __str__ → Name: Alice, Age: 25
print(len(p1)) # __len__ → 25
print(p1 + p2) # __add__ → 55

```

Top 5 Interview Dunders

1. `__init__` → constructor
1. `__str__` → string output
1. `__len__` → length
1. `__add__` → addition
1. `__call__` → make object callable like function

4. Generator

What is a Generator?

1. Function that **yields values one by one** (instead of returning all at once)
1. Saves memory (lazy execution)
1. Uses `yield` keyword

Simple Example

```
python
# Generator function
def count_up_to(n):
    count = 1
    while count <= n:
        yield count # pause & return value
        count += 1

# Use generator
gen = count_up_to(3)

print(next(gen)) # 1
print(next(gen)) # 2
print(next(gen)) # 3
```

Generator Expression (One-Liner)

```
python
gen = (x for x in range(3))
print(list(gen)) # [0,1,2]
```

Key: Generators = memory efficient for large data

5. Context Manager

What is it?

1. Automatically manages resources (open/close files, locks)
1. Uses `with` statement
1. Implemented with `__enter__` and `__exit__` OR `@contextmanager`

A. Class-Based Context Manager

```
python
class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode

    def __enter__(self):
        self.file = open(self.filename, self.mode)
```

```

        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()

# Usage (auto closes file)
with FileManager("test.txt", "w") as f:
    f.write("Hello")

```

B. Function-Based (Decorator) → EASY

```

python
from contextlib import contextmanager

@contextmanager
def open_file(name):
    f = open(name, 'w')
    yield f # give control to block
    f.close() # runs after block

# Usage
with open_file("test.txt") as f:
    f.write("Python!")

```

Why use it? No need to manually `close()` → prevents resource leaks

6. asyncio (Asynchronous Programming)

What is asyncio?

1. For **concurrent code** (no threads, single-threaded concurrency)
1. Use `async` (define) + `await` (wait)

Simple Example

```

python
import asyncio

# Async function
async def task(name, delay):
    print(f"Start {name}")

```

```
    await asyncio.sleep(delay) # non-blocking wait
    print(f"End {name}")

# Run multiple tasks
async def main():
    # Run concurrently
    await asyncio.gather(
        task("A", 2),
        task("B", 1)
    )

# Run main
asyncio.run(main())
```

Output:

```
text
Start A
Start B
End B # B finishes first (1s)
End A # A finishes later (2s)
```

Key Terms

1. `async def` → async function
1. `await` → wait for async task
1. `asyncio.run()` → run main function
1. `asyncio.gather()` → run multiple tasks at once

Quick Interview Cheat List (Memorize This!)

1. **Decorator:** `@` → add behavior to func/class
1. **Dunder:** `__method__` → `__init__`, `__str__`, `__len__`, `__add__`
1. **Generator:** `yield` → lazy, memory efficient
1. **Context Manager:** `with` → auto resource clean-up
1. **asyncio:** `async` + `await` → single-thread concurrency

| (注：文档部分内容可能由 AI 生成)